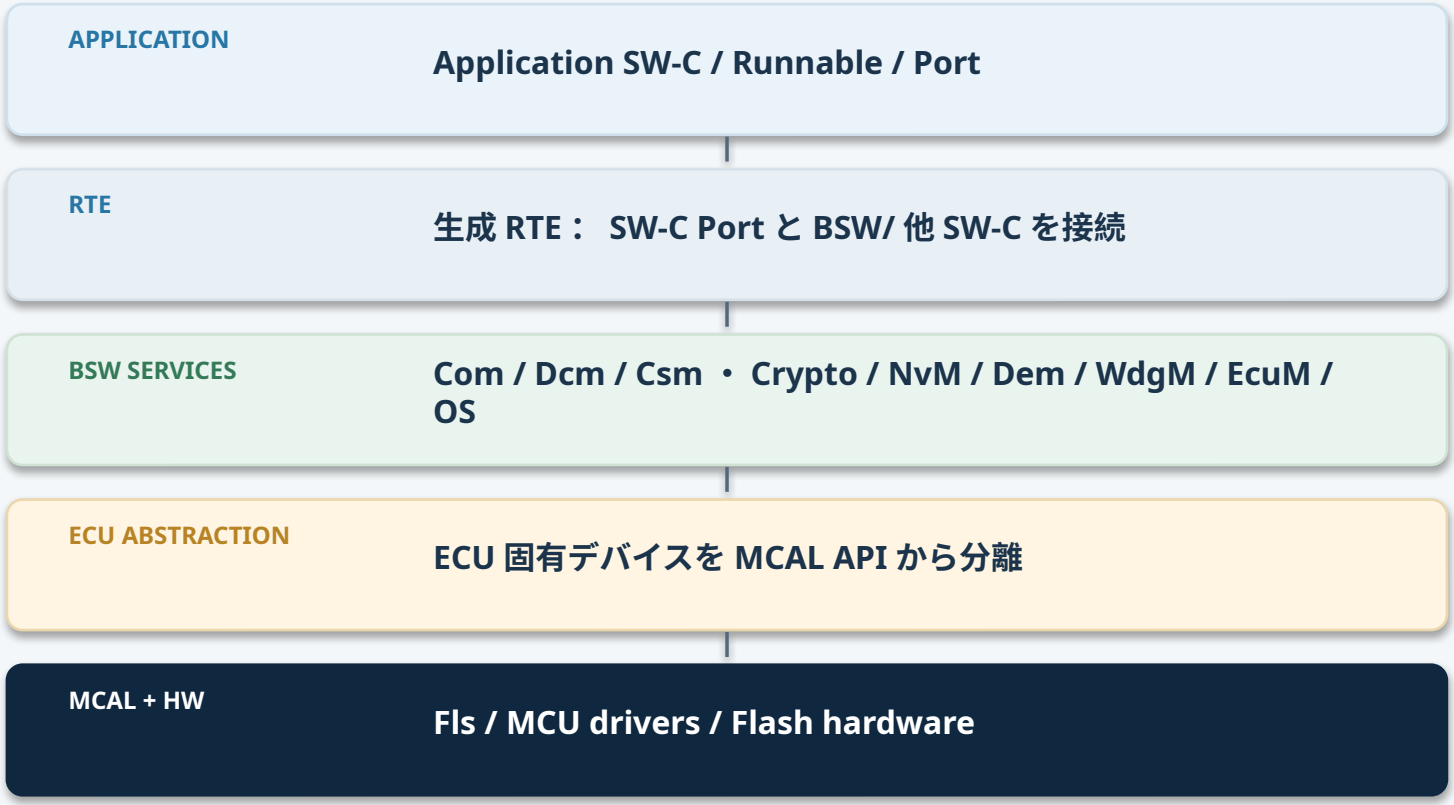
AUTOSAR Classic の層に責務を置く

上位の SW-C は RTE ポート契約に依存し、 BSW から下は標準サービス・ ECU 抽象化・ MCAL で車両 / ハードウェア差分を吸収する。



FOTA の配置

- VCU： 更新キャンペーン制御
- 対象 ECU： OEM 定義の更新 BSW サービス
- Bootloader： 起動バンクと復帰を独立管理

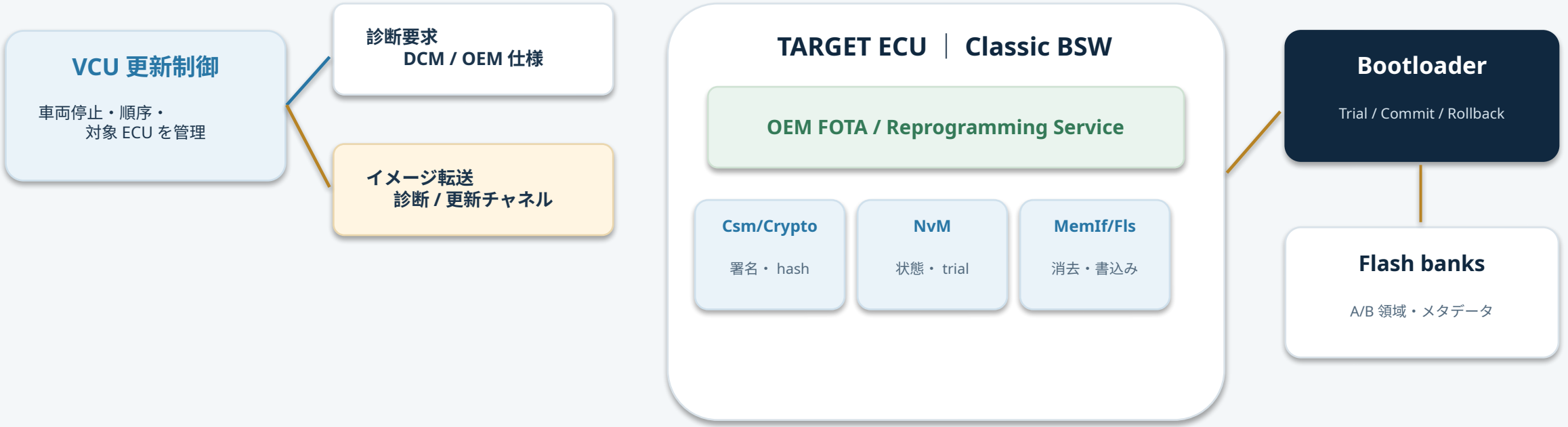
Pi 5 の扱い

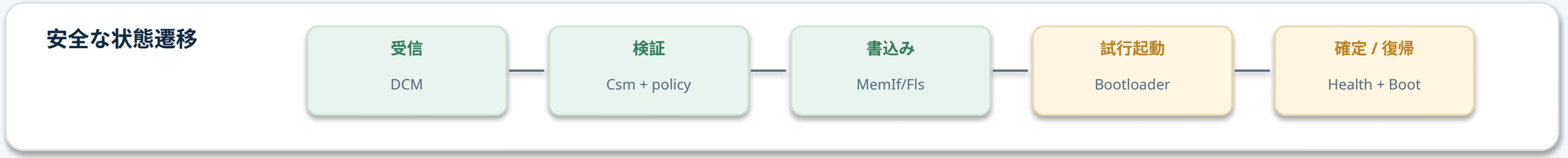
Linux adapter は開発・検証用。 Classic OS/BSW/MCAL ではない。実 Classic ECU 化には対象をサポートする stack が必要。

AUTOSAR Classic は RTE / BSW / ECU Extract / 生成設定まで含む統合構造。単に trait で抽象化するだけでは Classic 準拠にならない。

Classic ECU での更新フロー

FOTA 全体は OEM/BSW vendor の統合機能。 DCM ・暗号・メモリサービスと独立 Bootloader の責務を明確にする。





車両を載せ替えた場合の差分

再利用の単位は SW-C のポート契約。 RTE/BSW 設定と ECU 依存部を variant として再統合する。

変更シナリオ	維持できる資産	変更・再検証する資産
同じ ECU / 別車両	SW-C 内部ロジック ポート仕様が同じ範囲	ECU Extract 、 RTE 生成設定、 Com/ 診断設定、信号経路、校正、車両互換性表
同じ SW-C / 別 ECU HW	標準 RTE/BSW API に依存する機能	MCAL 、 ECU Abstraction 、 Fls/NvM 設定、 OS/ タスク、メモリ配置、Bootloader
別 BSW vendor / release	要求・状態遷移・データ契約	BSW API binding 、生成設定、 crypto/diagnostics/memory 統合、適合・回帰試験

Pi 5 上の Linux adapter を差し替えるだけで Classic ECU へ移行できるわけではない。 Classic stack / MCAL / Bootloader 製品に合わせた下位統合が必要。

現行 Rust PoC から Classic 統合へ

今ある抽象化は足場として活用し、 Classic 標準サービスとの接続・固定メモリ・生成設定へ段階的に寄せる。

現行コード

sdv-core： 独自 SW-C/RTE

sdv-fota： 独自 Rust 更新 service

ImageStorage / BootControl： 独自 trait

linux.rs： ファイル A/B + CRC metadata

serde / String / Vec / alloc 利用

Pi 5 実機・ AUTOSAR stack 未接続

Classic に向けた実装変更

1. SW-C/RTE 契約と更新 BSW 責務を分離
2. FOTA core を固定長 buffer ・ no-heap 化
3. Crypto / NvM / Fls / Boot API 境界を定義
4. Host Linux backend は開発専用に隔離
5. 採用 stack 決定後に C API/ 生成設定へ接続

先に決めること

- Classic stack / release
- 対象 ECU と OS/MCU
- DCM reprogramming 設計
- NvM / Flash / bank 配置
- Bootloader 責任境界
- Pi 5 を host で使う範囲